PSTAT 10 Data Science Principles

Lecture 8: Plotting

John Robin Inston
johninston@ucsb.edu
University of California, Santa Barbara

August 12, 2026

Introduction

🔁Review: Lecture 7

👈Last lecture we studied the following topics:

- Tibble Objects;
- Tibble Manipulation;
  - Selecting;
  - Filtering;
  - Mutating;
- Reading Data
  - `.txt`;
  - `.csv`; and
  - `.xlsx`.

👀Outline: Lecture 8

👇Today we will continue looking at **describing** and **visualizing** external data, specifically:

- Descriptive statistics
- Base R Plotting
- Probability
- Discrete and Continuous Random Variables

Exploring Real Data

`.csv` Files

File Types Recap

Last lecture we looked at several different file types that data can be stored as, including:

- `.txt` files
- `.csv` files
- `.xlsx` files

Why `.csv`?

In this course we shall be using `.csv` files almost exclusively, and so we typically will only need the `read.csv()` function to load real world data.

Loading `Michelson.csv`

The `Michelson` data is a famous dataset measuring the speed of light. I keep my data in a `data` subfolder, hence the path name below.

```
1 Michelson <- read.csv("data/Michelson.csv")
2 head(Michelson, 3)
```

```
  ds12 ds13 ds14 ds15 ds16
1  850  740  900 1070  930
2  850  950  980  980  880
3 1000  980  930  650  760
```

Messy Data Problems

The Reality of Real Data

Real data is rarely clean and well behaved. It is often **messy** and **unintuitive**, and it is our job as data scientists and statisticians to **understand**, **interpret** and **draw meaning** from the mess.

Key Tools

To accomplish this goal we begin with several key tools:

- Summary Statistics
- Graphical Tools
- Statistical Measures (Correlation, Standard Deviation)
- Prior Knowledge
- Qualitative Analysis (understanding where the data comes from)

📖Statistics Terminology

Population & Sample

Data consists of information from observations, counts, measurements or responses.

- The population is the complete collection of all observations.
- The sample is a representative group selected from a population.

Parameters & Statistics

- A parameter describes a population characteristic; a statistic describes a sample characteristic.
- Descriptive statistics summarize data.
- Inferential statistics test hypotheses (PSTAT 126).

📊 Summary Statistics

Summary statistics are the key statistics of a sample that we always generate to get an idea about the shape and location of the data.

Key Statistics

1. Minimum
2. Lower Quartile
3. Median
4. Mean
5. Upper Quartile
6. Maximum
7. Spread Measures (Standard Deviation, Variance)
8. Outliers
9. Variable Correlation

💪Exercise — Summary Statistics

The `summary()` function provides the first 6 statistics. The correlation and standard deviation are given by the `cor()` and `sd()` functions respectively.

1. Change the `Michelson` object to a tibble using the function `tibble()`.
2. Select the column `ds16` using the `select()` function and apply the function `summary()`.

02:00

✅Solution — Summary Statistics

- Convert `Michelson` to a tibble using `tibble()`.
- Select the `ds16` column using `select()`.
- Apply `summary()` to obtain the key summary statistics.

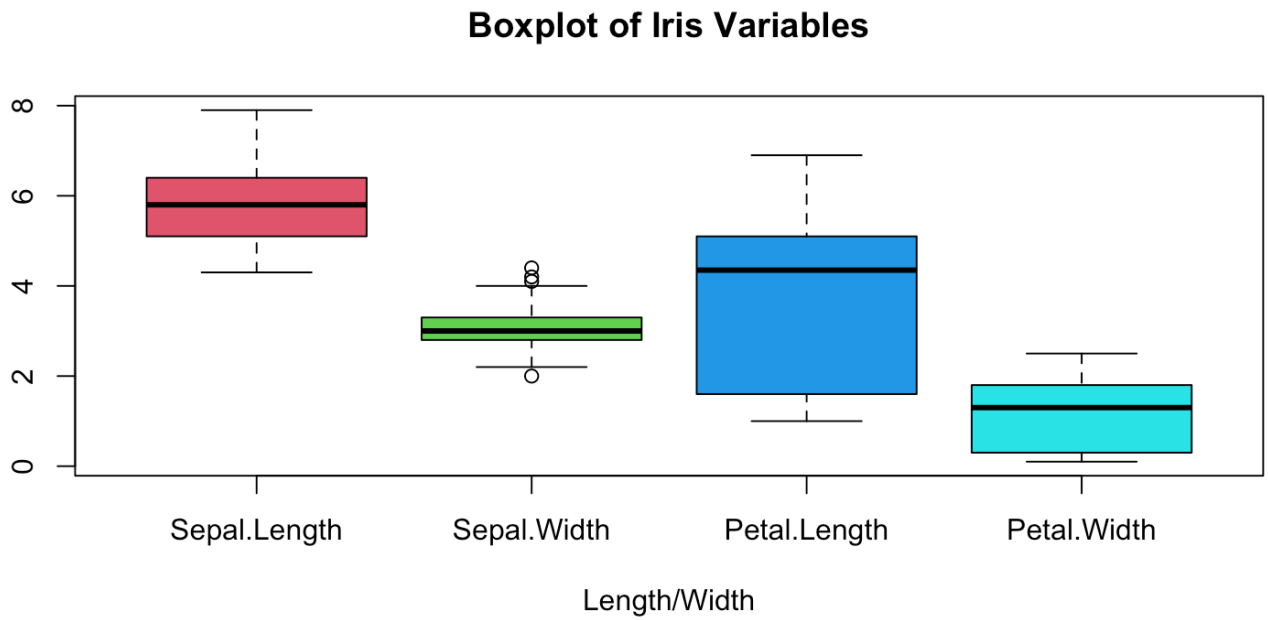
```
1 Michelson <- Michelson |> tibble()
2
3 Michelson |>
4   select(ds16) |>
5   summary()
```

```
      ds16
 Min.   :720.0
 1st Qu.:795.0
 Median :840.0
 Mean   :839.5
 3rd Qu.:880.0
 Max.   :960.0
```

Boxplots

Boxplots are a graphical visualization of most of the information provided by the `summary()` function.

```
1 boxplot(iris[,-5], col = 2:5, xlab="Length/Width",
2         main="Boxplot of Iris Variables")
```



📈Descriptive Statistics

Descriptive statistics help to summarize key characteristics of the data such as:

- its location
- its spread
- if its distribution has one peak (unimodal) or many peaks
- how sharp these peaks are (kurtosis)
- whether the data is skewed one way or the other

> 📌In this course we only consider location and spread.

⚖️Measures of Centrality & Spread

Measures of Centrality

- Arithmetic average or mean — use the function `mean()`
- Data midpoint, a.k.a. the median — use the function `median()`
- Most frequent value, a.k.a. the mode — use the function `mode()`

Measures of Spread

- Range (difference between maximum and minimum) — use the function `range()`
- Interquartile Range (difference between upper and lower quartile)
- Variance and Standard Deviation — use the functions `var()` and `sd()`

🧮 Sample Mean & Standard Deviation

Sample Mean

The sample mean is just the arithmetic average. For a sample $(x_1, \ldots, x_n)$ the sample mean $\bar{x}$ is given by

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

> 📌 This is only an estimate of the (true) population mean, which we denote μ.

Sample Standard Deviation

The sample standard deviation s is:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

> 📌 This estimates the (true) population standard deviation σ (recall σ^2 is the variance).

🔗Relation Measures

> 📌**Relation measures** describe how **two variables change together** — i.e. how does one variable change if we change the value of another?

Intuition

- Both **increase** together → **positive relationship** (e.g. height & weight).
- One **increases** as the other **decreases** → **negative relationship** (e.g. price & demand).
- No consistent pattern → **unrelated** (e.g. shoe size & IQ).

Formal Measures

Covariance measures the joint variability of two variables, and leads to **correlation** — a measure of the strength of their linear relationship.

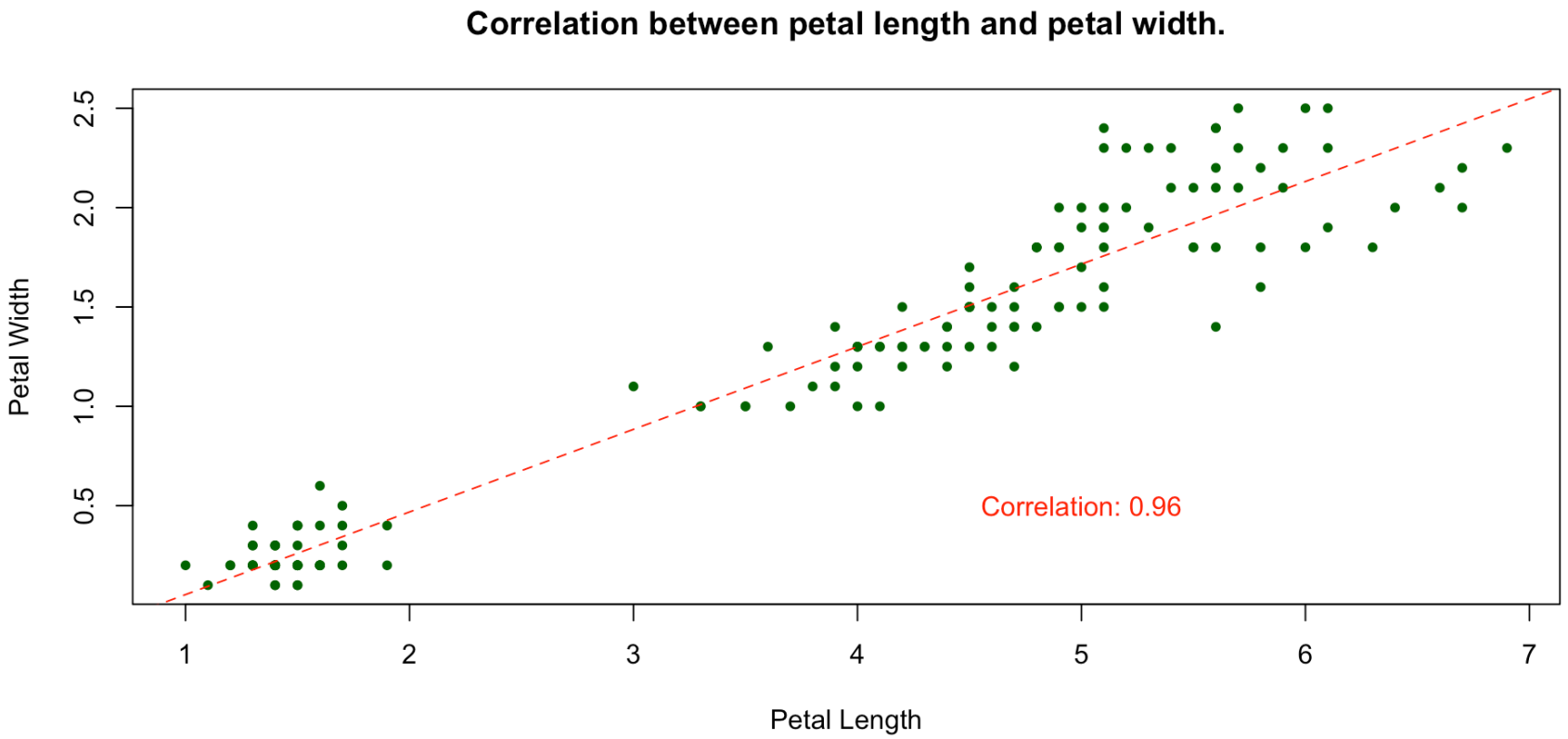
```
cov(iris$Petal.Length, iris$Petal.Width)
```

```
[1] 1.295609
```

```
cor(iris$Petal.Length, iris$Petal.Width)
```

```
[1] 0.9628654
```

📈 Example — Correlation Plot



Scatter plot with line of best fit.

Covariance & Correlation

Covariance

The covariance between two variables $x := (x_1, \ldots, x_n)$ and $y := (y_1, \ldots, y_n)$ is given by

$$\text{Cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

> 📌The covariance can be negative, which indicates a negative linear relationship (one goes up means the other goes down).

Correlation

From the definition of covariance we naturally define correlation, denoted by the Greek letter rho ρ, of two variables x and y as

$$\rho_{XY} = \frac{\text{Cov}(x, y)}{s_x \cdot s_y}$$

We can compute the correlation between two vectors in R using the function `cor()`.

💪Exercise — Computing Summary Statistics

We shall be considering the `faithful.csv` dataset, which contains a list of waiting times between eruptions and the duration of eruptions for the Old Faithful geyser in Yellowstone National Park.

1. Download the data from Canvas and save it to your working directory.
2. Load the data into R and save it to the object `faithful`. Convert this to a `tibble()` object.
3. Using the functions `mean()` and `sd()`, compute the mean of `eruptions` and the standard deviation of `waiting`.
4. Use the functions `cov()` and `cor()` to compute both the covariance and the correlation between `eruptions` and `waiting`.

05:00

✅Solution — Computing Summary Statistics

- **Load** `faithful.csv` and convert it to a tibble.
- **Compute** the mean of `eruptions` and the standard deviation of `waiting` using `mean()` and `sd()`.
- **Compute** the covariance and correlation between `eruptions` and `waiting` using `cov()` and `cor()`.

```
 1  # load data
 2  faithful <- read.csv("data/faithful.csv") |> tibble()
 3  # mean
 4  faithful$eruptions |> mean()
 5  # standard deviation
 6  faithful$waiting |> sd()
 7  # covariance
 8  cov(faithful$eruptions, faithful$waiting)
 9  # correlation
10  cor(faithful$eruptions, faithful$waiting)
```

```
[1] 3.487783
```

```
[1] 13.59497
```

```
[1] 13.97781
```

```
[1] 0.9008112
```

⚠️Problems with Descriptive Statistics

The Problem

It is very important to practice **caution** when using summary and descriptive statistics. They are very **simplistic** and are easily fooled by more complex data patterns.

Example Data

To demonstrate some of the problems you may run into we consider `problem.csv`, which can be found on Canvas.

```
1 # load problem data
2 problem <- read.csv("data/problem.csv") |>
3   tibble()
4 # inspect data
5 problem |> head(8)
```

```
# A tibble: 8 × 4
      x     y     a     b
  <int> <int> <int> <dbl>
1     5    45     1 0
2     2    50     2 0.788
3     5    47     3 1.10
4     5    46     4 1.39
5     0    46     5 1.61
6     5    55     6 1.79
7     1    54     7 1.95
8     0    46     8 2.08
```

⚠️Interpreting the Summary

Applying `summary()` to `problem` we obtain the following summary statistics:

```
1 summary(problem)
```

```
       x                y                a               b        
 Min.   :  0.00   Min.   :   41.0   Min.   : 1.00   Min.   :0.000  
 1st Qu.:  2.25   1st Qu.:   49.0   1st Qu.:12.25   1st Qu.:2.505  
 Median : 46.50   Median :   50.5   Median :21.50   Median :3.068  
 Mean   : 48.87   Mean   :  405.3   Mean   :22.37   Mean   :2.855  
 3rd Qu.: 97.75   3rd Qu.:   53.0   3rd Qu.:32.75   3rd Qu.:3.489  
 Max.   :100.00   Max.   :16368.0   Max.   :44.00   Max.   :3.784  
```

We might make the following quick observations:

- The variable `x` is clustered around 48.
- The variable `y` is clustered around 405.

```
1 cor(problem$a, problem$b)
```

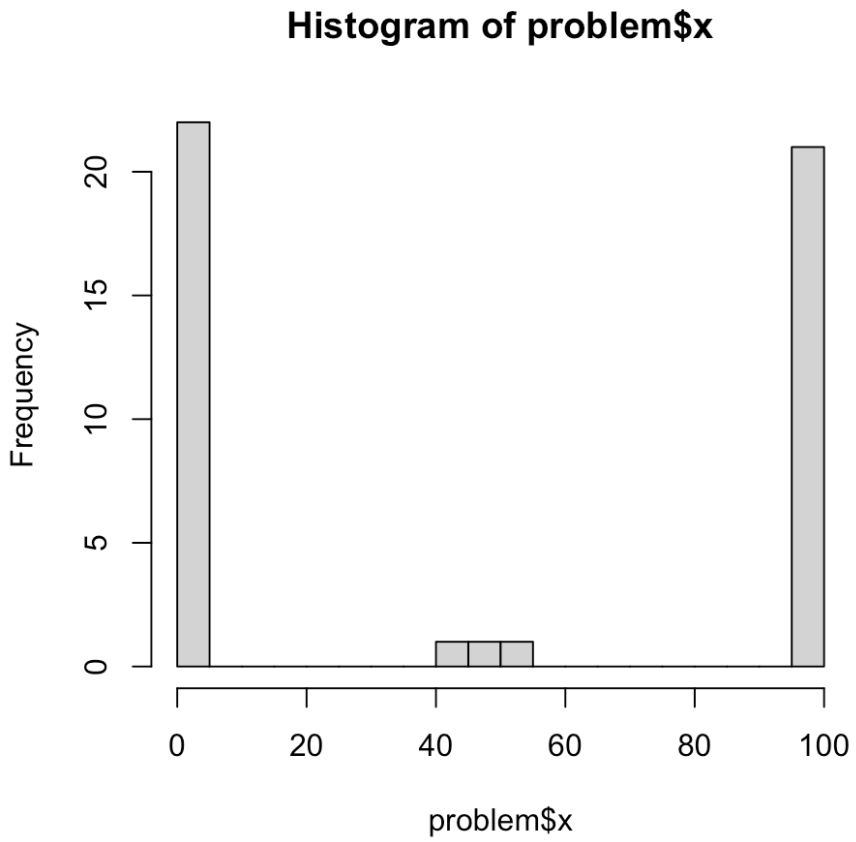
```
[1] 0.9102097
```

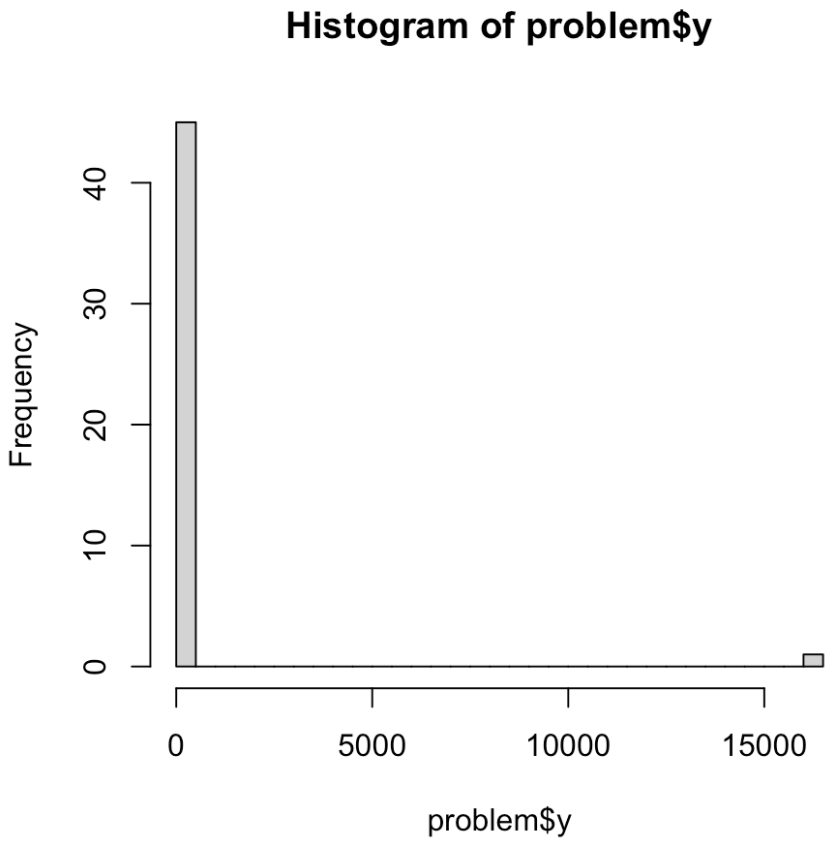
Furthermore we might conclude that there is a significant linear relationship between variables `a` and `b`, as their correlation is 0.91.

⚠️A Closer Look — Histogram

A closer look at `x` and `y`:

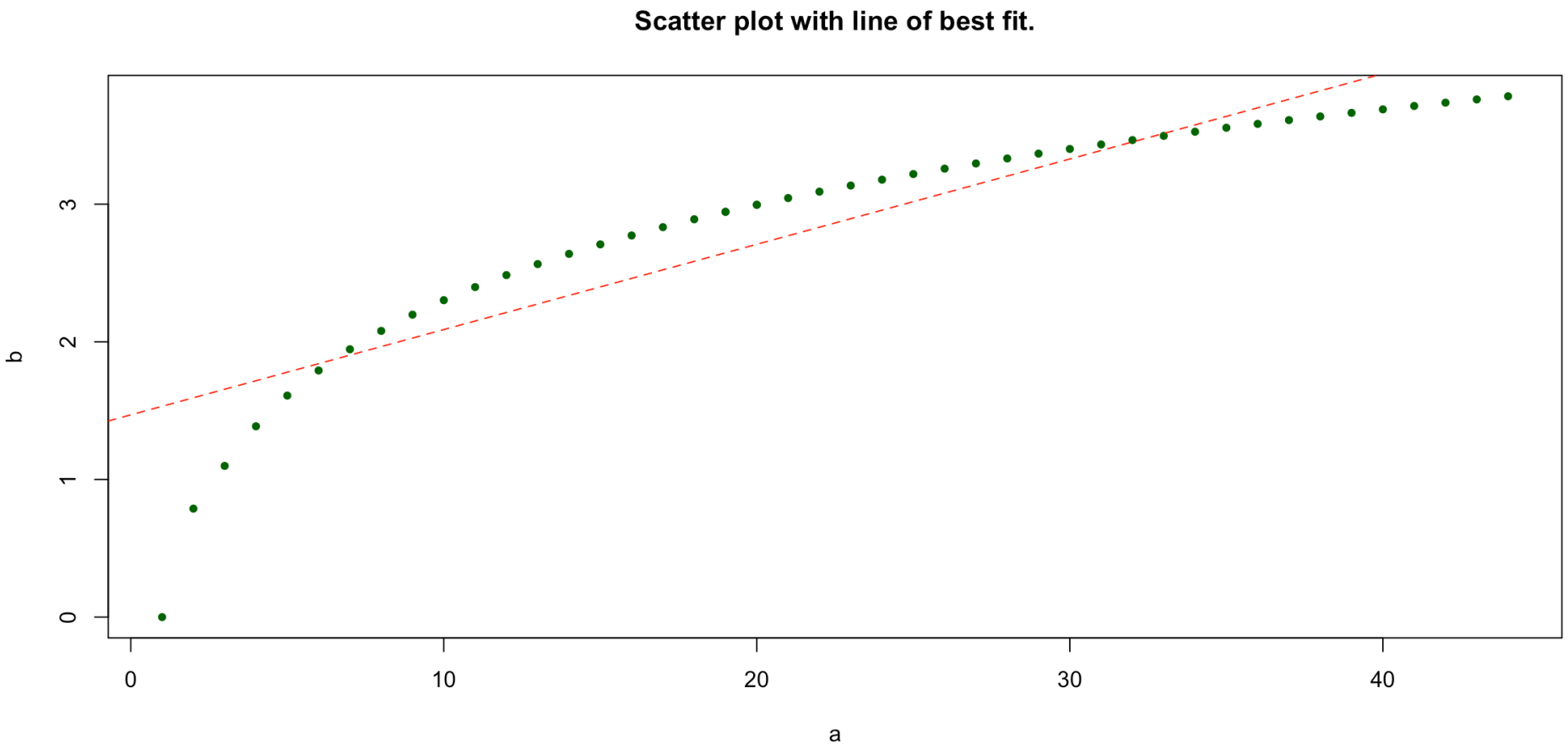
```
1 par(mfrow = c(1,2))
2 hist(problem$x, breaks = 30)
3 hist(problem$y, breaks = 30)
```





⚠️A Closer Look — Scatter Plot

The relationship between `a` and `b` does not appear to be linear!

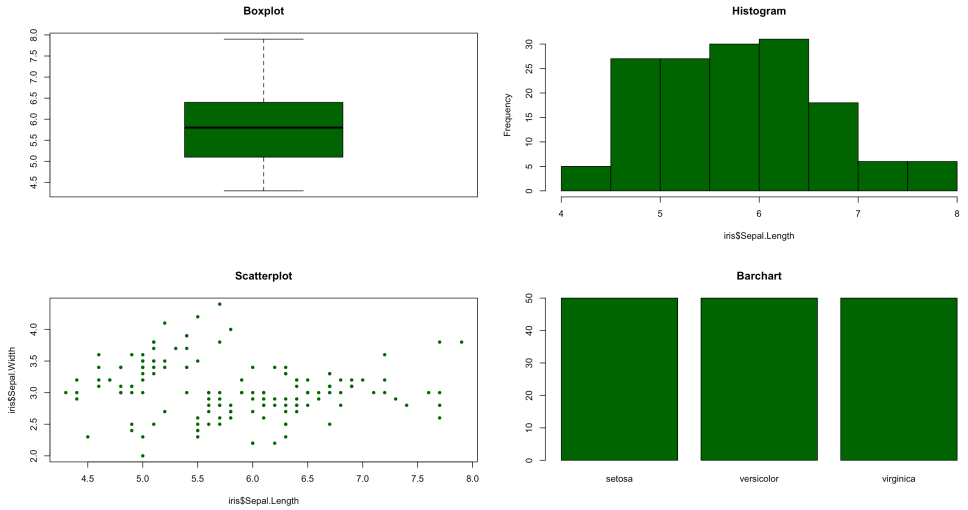


Plotting in Base R

🖼️Types of Plots

There are a wide variety of plots which all have their uses for data visualization and interpretation, including:

- Scatterplots
- Boxplots
- Histograms
- Barcharts
- Pie Charts
- Q-Q plots



Example Data

For an example we consider the `airquality` data in the `datasets` library.

```
1  # load data
2  airquality <- datasets::airquality |>
3    tibble()
4  airquality |> head(3)
```

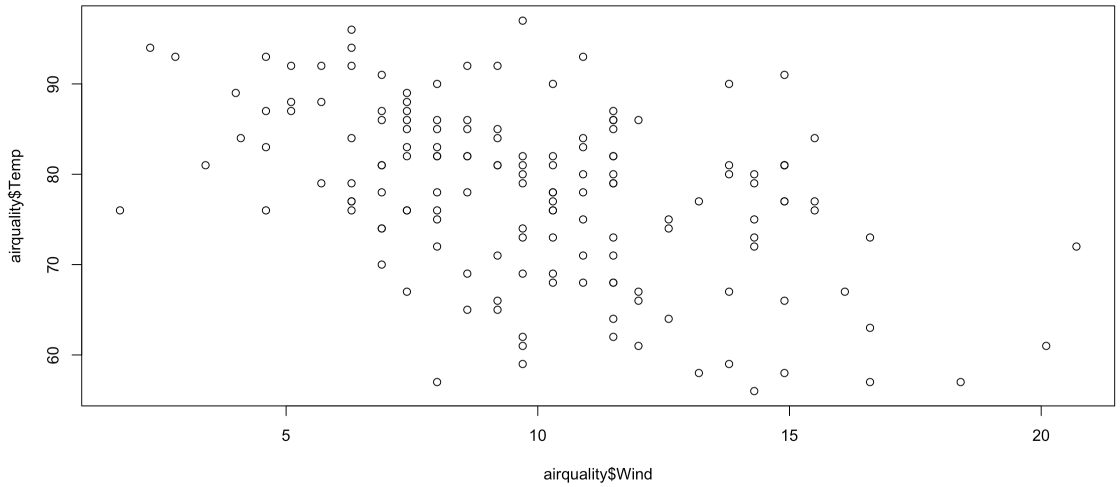
```
# A tibble: 3 × 6
  Ozone Solar.R  Wind  Temp Month   Day
  <int>   <int> <dbl> <int> <int> <int>
1    41     190   7.4    67     5     1
2    36     118   8      72     5     2
3    12     149  12.6    74     5     3
```

😬 An "Unprofessional" Plot

A Basic Scatter Plot

We produce a scatter plot of `Wind` against `Temp` using the `plot()` function.

```
1 plot(airquality$Wind, airquality$Temp)
```



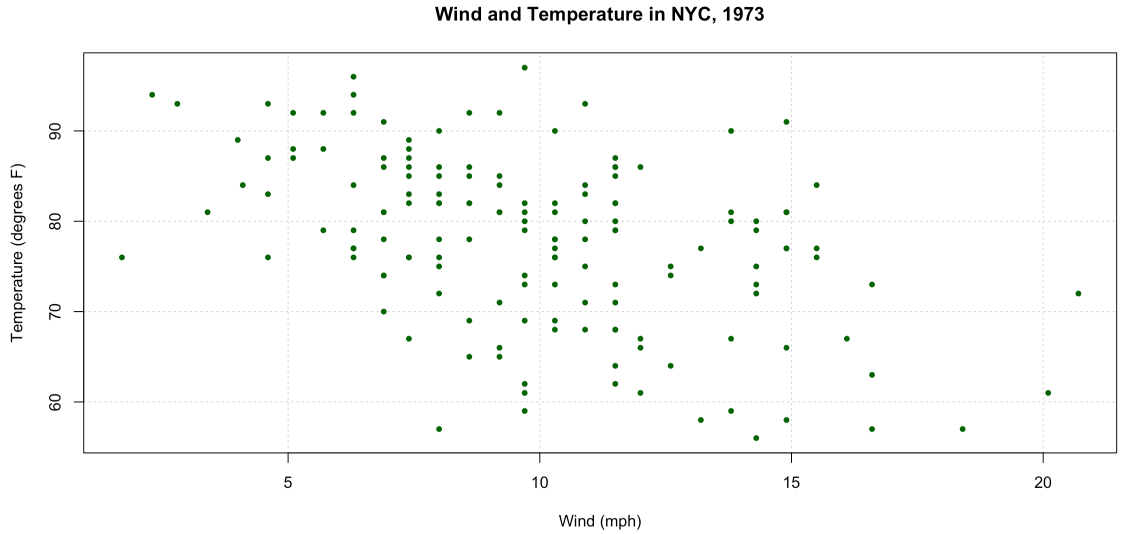
A very basic scatter plot!

- The axes are not labelled and there is no title.
- There is no formatting.

🎨A More Professional Plot

This is fine but not very professional. Fortunately, `plot()` has a lot of arguments we can use to tune our plot.

```
plot(airquality$Wind, airquality$Temp,
  main = "Wind and Temperature in NYC, 1973",
  xlab = "Wind (mph)",
  ylab = "Temperature (degrees F)",
  pch = 20,
  col = "darkgreen",
  panel.first = grid()
)
```



A more professional plot!

🎨Further Improvements

What We Changed

- Defined a plot **title** using `main="..."`;
- Defined x and y **axis labels** using `xlab="..."` and `ylab="..."`;
- Formatted our **point type** to be solid `pch=20` and dark green `col="darkgreen"`.

Line of Best Fit

We also might want to overlay a **line of best fit** using the **linear model** `lm()` function.

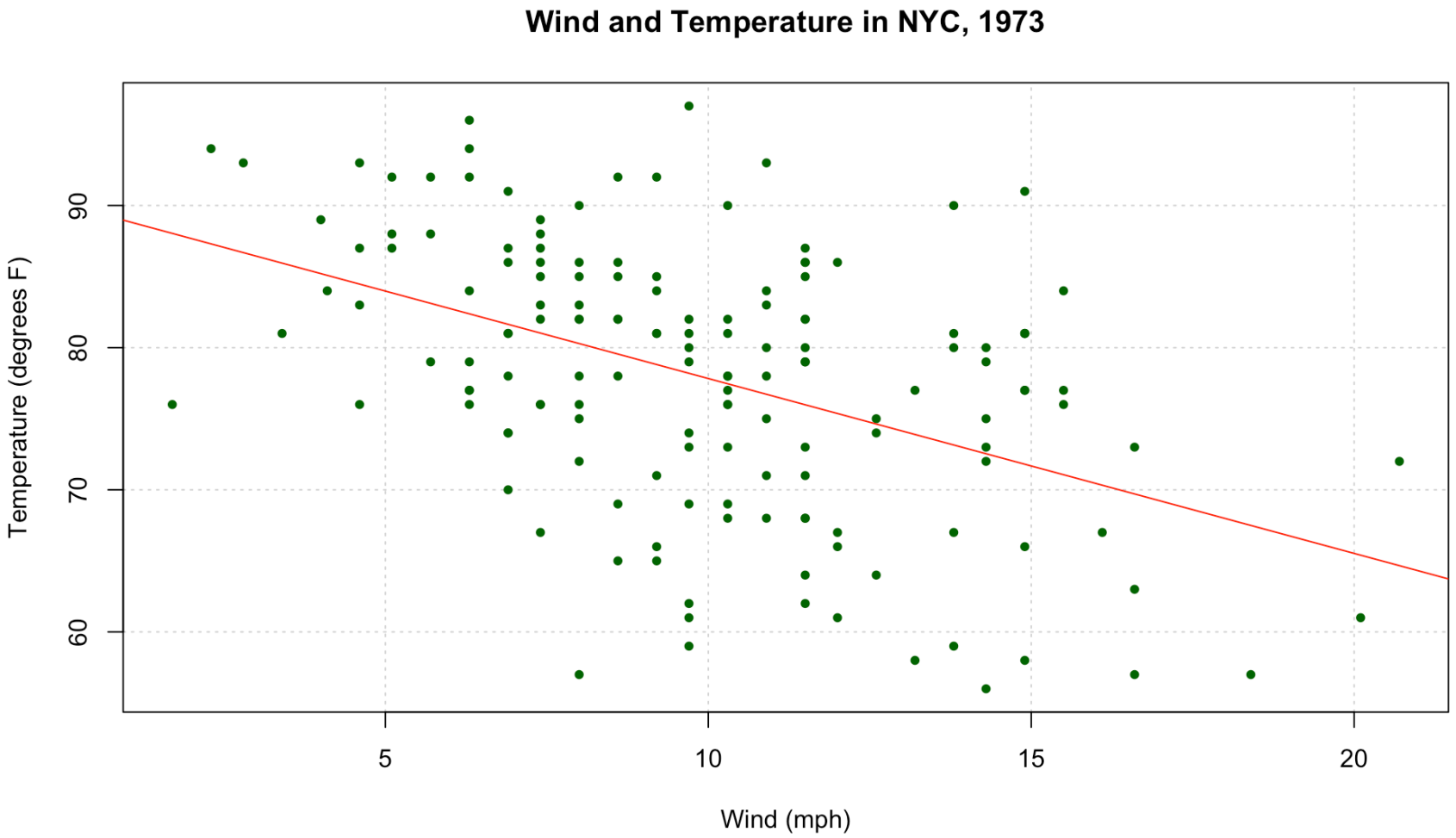
We will not cover the `lm()` function in this course, but in summary it fits a linear regression model between the `x` and `y` variables and we use this model to plot the line of best fit.

We add this line to our plot using the function `abline()`.

📉 Line of Best Fit

```r
# A more professional plot with best fit line
plot(airquality$Wind, airquality$Temp,
  main = "Wind and Temperature in NYC, 1973",
  xlab = "Wind (mph)",
  ylab = "Temperature (degrees F)",
  pch = 20,
  col = "darkgreen",
  panel.first = grid()
)
abline(lm(airquality$Temp ~ airquality$Wind), col = "red")
```

Line of Best Fit



A more professional plot with a line of best fit!

📐 Formula Notation

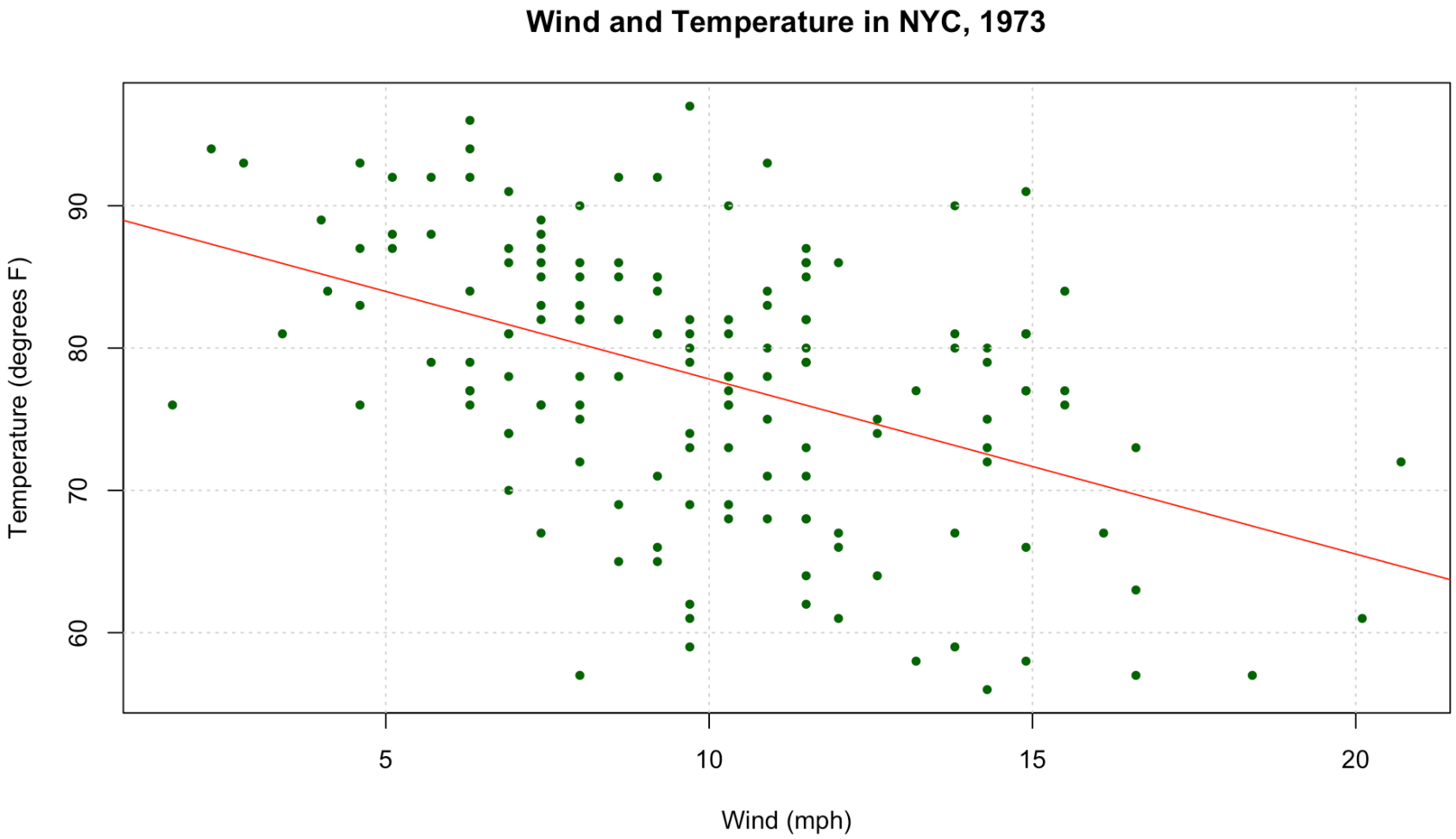
Notice in `lm()` we used R's **formula notation** `y ~ x`. It is recommended to use formula notation for your plots as it makes your code clearer.

Key Syntax

- `y ~ x` reads as "**y modeled by x**" — the **response** goes on the left of `~`, **predictors** go on the right.
- The `data = ...` argument lets you refer to columns **by name** directly, without repeating `dataframe$` in front of every variable.
- Multiple predictors can be combined with `+`, e.g. `y ~ x1 + x2`.

```r
# A more professional plot with best fit line
plot(Temp ~ Wind, data = airquality,
  main = "Wind and Temperature in NYC, 1973",
  xlab = "Wind (mph)",
  ylab = "Temperature (degrees F)",
  pch = 20,
  col = "darkgreen"
)
grid()
abline(lm(airquality$Temp ~ airquality$Wind), col = "red")
```

Formula Notation



A more professional plot with a line of best fit!

〰Plotting Continuous Functions

Discretizing

- Computers think in **discrete terms** and cannot represent **continuous time**.
- To plot continuous functions, e.g. `sin()` or `exp()`, we **discretize** them — redefining the interval as a sequence with **very fine discrete jumps**.
- For example, on the interval $[0, 10]$ we could define:

```r
x <- seq(0, 10, by = 0.01)
head(x)
```

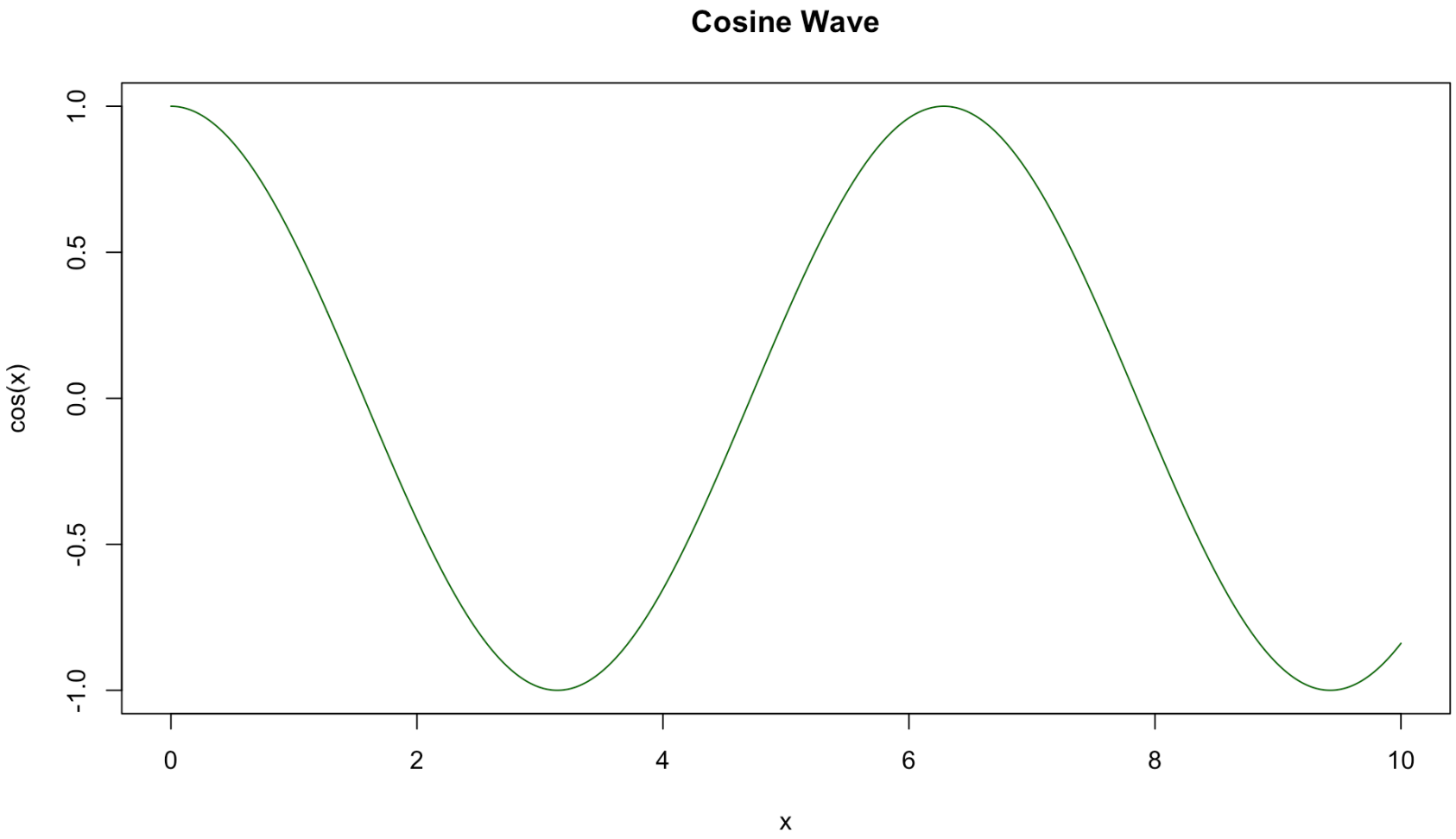
```
[1] 0.00 0.01 0.02 0.03 0.04 0.05
```

Cosine Wave Example

- We compute `cos(x)` and specify `type="l"`, which plots a **smooth line** between the points rather than the points themselves.

```r
plot(x, cos(x),
  main = "Cosine Wave",
  xlab = "x",
  ylab = "cos(x)",
  col = "darkgreen",
  type = "l"
)
```

〰Plotting Continuous Functions



Cosine Wave

➕Overlaying Lines

Using `lines()`

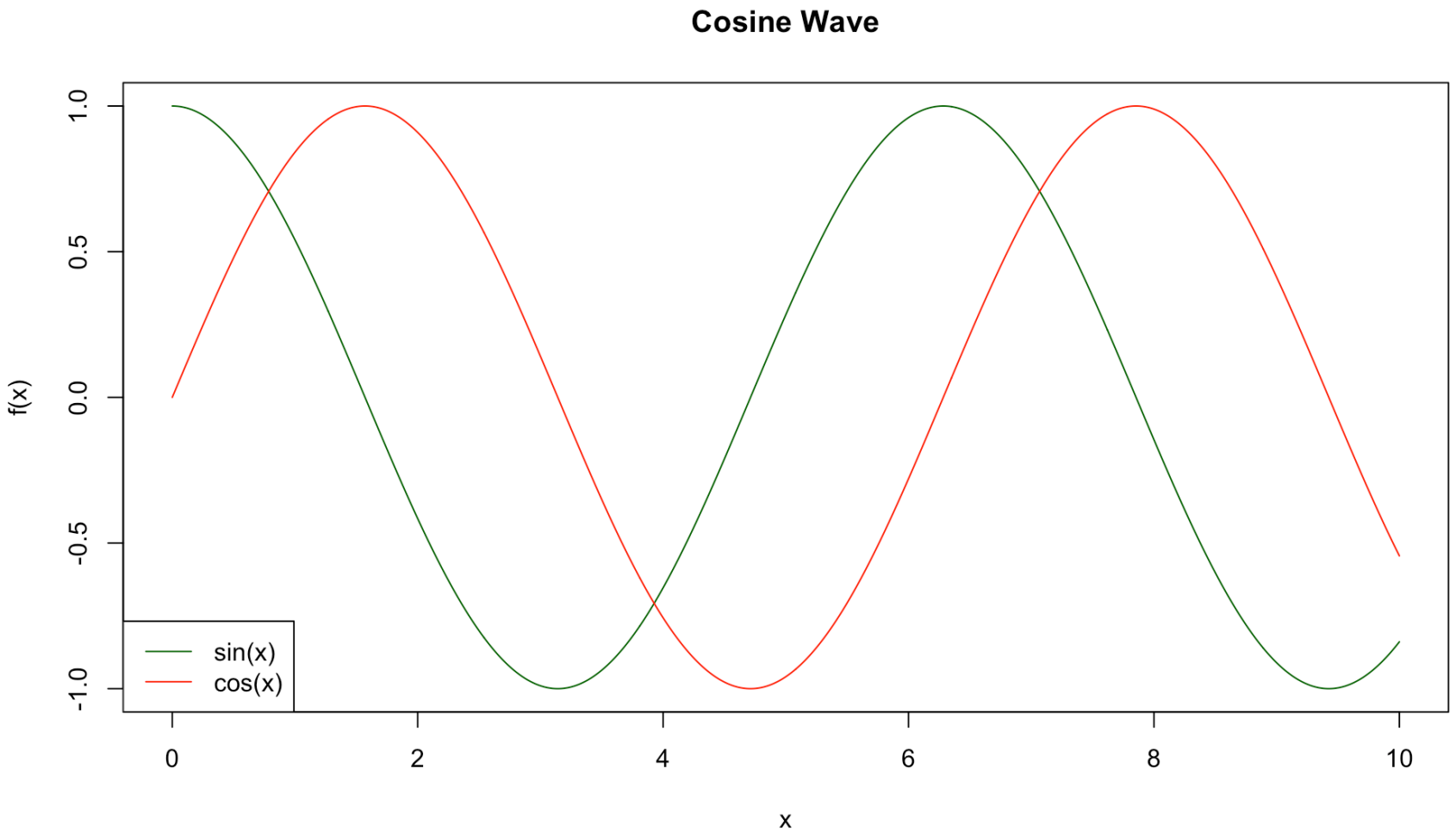
- We can overlay several lines one on top of the other using the `lines()` function.
- `lines()` takes broadly the same arguments as `plot()` (e.g. `col`, `type`, `lty`).

Adding a Legend

- We add a legend using the function `legend()`.
- The first argument (e.g. `"bottomleft"`) sets the position of the legend.
- `legend = c(...)` gives the labels for each line.
- `col = ...` and `lty = ...` should match the plotted lines so the legend correctly identifies each one.

```
# A more professional plot with best fit line
plot(x, cos(x),
  main = "Cosine Wave",
  xlab = "x", ylab = "f(x)",
  col = "darkgreen", type = "l"
)
lines(x, sin(x), col = "red")
legend("bottomleft",
  legend = c("sin(x)", "cos(x)"),
  col = c("darkgreen", "red"), lty = 1)
```

✚Overlaying Lines



Cosine & Sin Waves

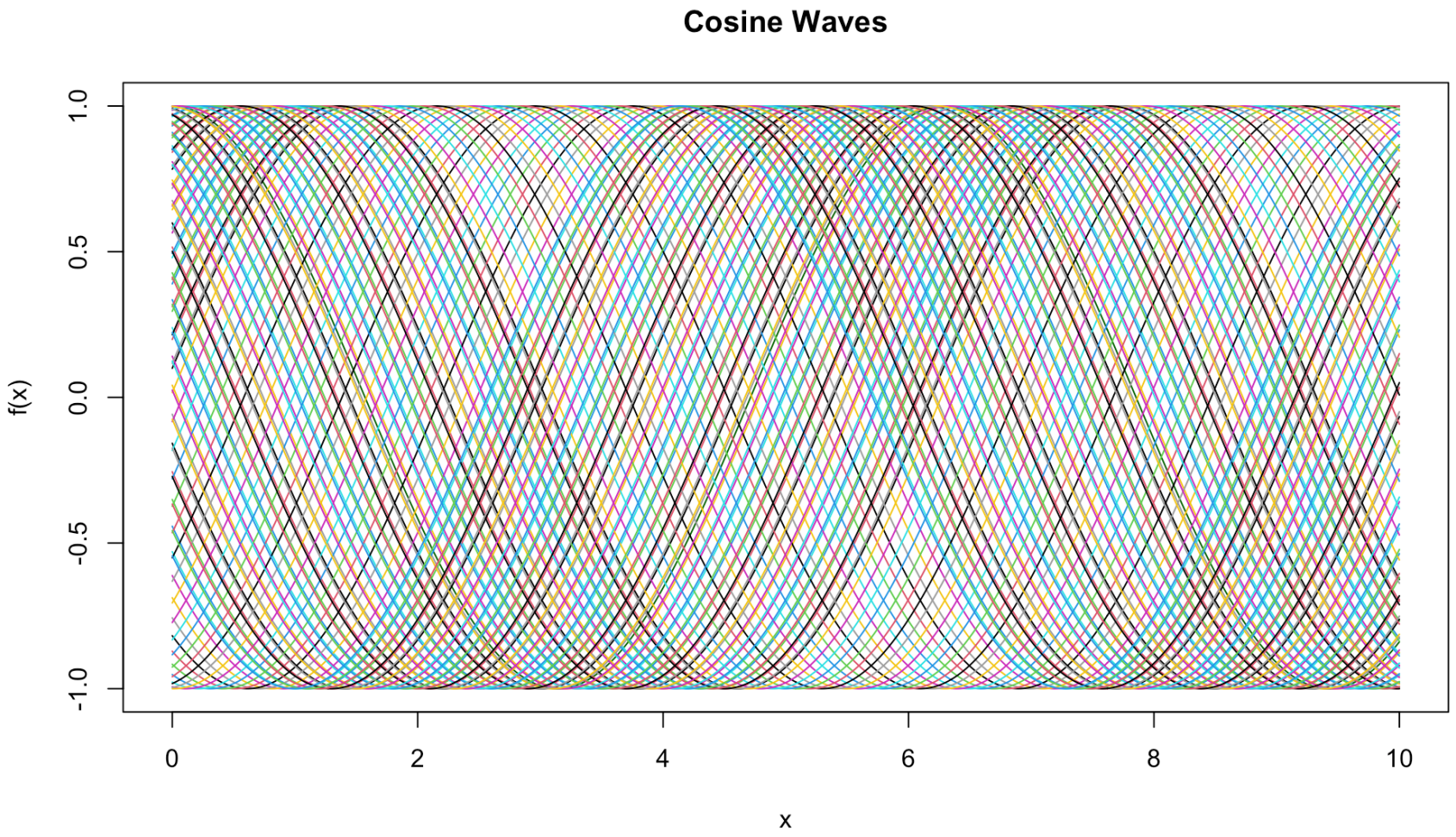
🔁 Iterating Function Plotting

For a bit of fun we conclude by demonstrating how a `for` loop can be used to iteratively produce complicated plots:

- We first plot the **base cosine wave**, then loop `i` from 1 to 100.
- Each iteration draws a **phase-shifted sine wave** `sin(x + i/10)` using `lines()`.
- Setting `col = i` cycles through R's **default color palette**, giving each new line a different color.
- The result is a dense fan of overlapping waves, all built up from a **single simple plotting call repeated** inside the loop.

```r
# A more professional plot with best fit line
plot(x, cos(x),
  main = "Cosine Waves",
  xlab = "x",
  ylab = "f(x)",
  col = "darkgreen",
  type = "l"
)
for(i in 1:100){
  lines(x, sin(x+(i/10)),
  col = i)
}
```

🔁Iterating Function Plotting



Lots and lots of waves!

💪Exercise — Plotting in R

Produce plots for x^3 and e^x for `x` between 1 and 10.

1. Define your `x` as a discretized sequence of values.
2. Define `x_cubed` as x^3 and `exp_x` as e^x.
3. Construct a plot of `x` against `x_cubed`, adjusting formatting as desired.
4. Add an additional line to the plot representing e^x.
5. Add an appropriate legend.

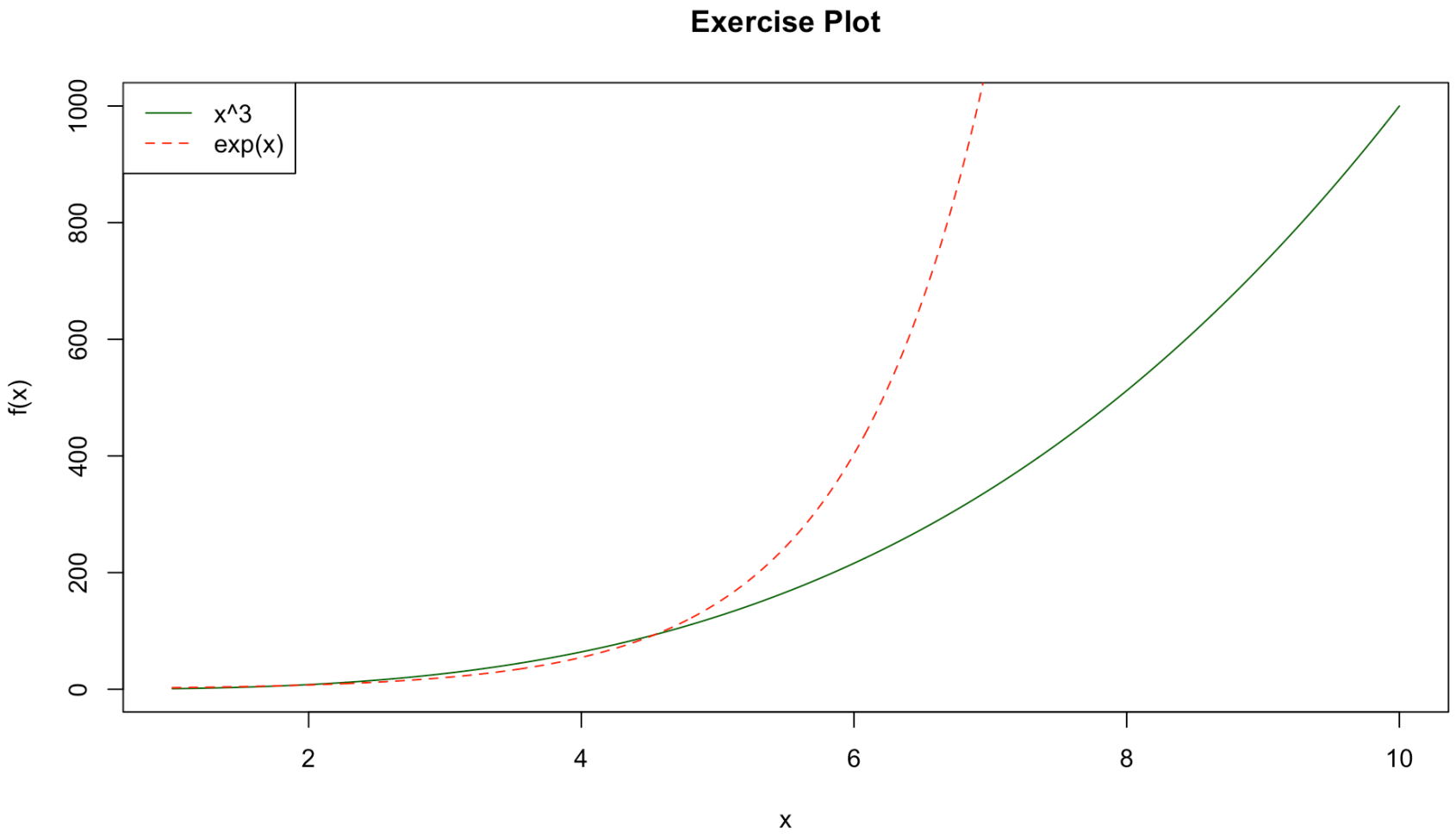
05:00

✅Solution — Plotting in R

- Discretize `x` between 1 and 10, then define `x_cubed` as x^3 and `x_exp` as e^x.
- Plot `x` against `x_cubed` as a line, then overlay `x_exp` using `lines()`.
- Add a legend identifying each line.

```r
# define variables
x <- seq(1, 10, by = 0.01)
x_cubed <- x^3
x_exp <- exp(x)
# produce plot
plot(
  x, x_cubed,
  main = "Exercise Plot",
  xlab = "x", ylab = "f(x)",
  type = "l", col = "darkgreen"
)
lines(x, x_exp, col = "red", lty = 2)
legend(
  "topleft",
  legend = c("x^3", "exp(x)"),
  col = c("darkgreen", "red"), lty = 1:2
)
```

✅Solution — Plotting in R



Exercise Plot

Summary

✅Topics Covered

🤔Today we looked into:

- Descriptive statistics
- Base R Plotting

📅 Next Class

😆Next class we will temporarily leave behind data science topics as we study probability theory! The next four classes will be (roughly) split as follows:

1. Introduction to Probability
2. Discrete Random Variables
3. Continuous Random Variables
4. Monte Carlo Methods